

```

# =====
# BLOB Principe - SAUT 2 : Q3 comme fleche du temps thermodynamique
# =====
#
# Objectif scientifique :
#   Caracteriser rigoureusement les proprietes mathematiques de Q3 = somme
#   |Delta S| sur les pas acceptes, pour determiner si elle satisfait les
#   axiomes d une fleche du temps thermodynamique.
#
# Proprietes a tester (par seed) :
#   P1 - Monotonie large : Q3 croissante ou constante (trivial, par construction)
#   P2 - Monotonie STRICTE :  $dQ3/dt > 0$  sur tous les intervalles non-triviaux
#   P3 - Comportement asymptotique : Q3 tend vers une limite finie ou infinie ?
#   P4 - Decroissance de la derivee :  $d^2 Q3 / dt^2 < 0$  (ralentissement) ?
#   P5 - Coherence avec T1 (temps emergent) : correlation Q3(t) vs T_n(t)
#
# Methode :
#   Re-executer le Blob 2.0 v2 sur 5 seeds (les memes que Saut 1) en
#   instrumentant beaucoup plus finement Q3 (echantillonnage dense + etat
#   du graphe a chaque seuil).
#
# Criteres de validation [M] :
#   1. P1 monotonie large : 5/5 seeds (tautologique, sert de check)
#   2. P2 monotonie stricte : 5/5 seeds, zero plateau prolonge
#   3. P4 ralentissement :  $d^2 Q3/dt^2 < 0$  en moyenne sur la 2e moitie
#   4. P5 correlation Q3 vs T_n > 0.95 sur 5/5 seeds
#
# Critere [T] candidat :
#   Si  $dQ3/dt > 0$  partout strictement, on peut prouver formellement que
#   Q3 est une fleche du temps au sens H de Boltzmann.
#
# Criteres de falsification :
#   -  $dQ3/dt = 0$  sur un intervalle non-trivial (>1% de la trajectoire)
#   - Q3 saturee ( $dQ3/dt \rightarrow 0$  avant fin de simulation)
#   - Correlation Q3 vs T_n < 0.5
# =====

import sys
import os

# =====
# AUTO-TEST CORRUPTION
# =====

def auto_check():

```

```

print("=" * 70)
print("  AUTO-TEST CORRUPTION CARACTERES")
print("=" * 70)
try:
    if hasattr(sys, 'argv') and len(sys.argv) > 0 and os.path.exists(sys.argv
        source_file = sys.argv[0]
    else:
        source_file = __file__ if '__file__' in globals() else None
    if source_file and os.path.exists(source_file):
        with open(source_file, 'r', encoding='utf-8') as f:
            content = f.read()
        dangerous = {
            chr(8216): "U+2018", chr(8217): "U+2019",
            chr(8220): "U+201C", chr(8221): "U+201D",
            chr(8211): "U+2013", chr(8212): "U+2014",
        }
        found = [(name, content.count(c)) for c, name in dangerous.items() if
            if found:
                print("ERREUR caracteres corrompus :", found)
                return False
        star = chr(42)
        bad_init = star + star + "init" + star + star
        if content.count(bad_init) > 1:
            print("ERREUR : motif corrompu init")
            return False
        triple_back = chr(96) * 3
        if content.count(triple_back) > 1:
            print("ERREUR : Markdown")
            return False
        print("OK : pas de corruption")
    else:
        print("Note : auto-test partiel")
except Exception as e:
    print("Note : " + str(e))
print("=" * 70)
return True

if not auto_check():
    sys.exit(1)

# =====
# IMPORTS
# =====
import numpy as np
import time
import json

```

```

import gc
from collections import Counter

# =====
# Parametres canoniques
# =====
KAPPA_0_TARGET = 8.0
BETA = 0.30
TAU = 0.50

P_MITOSE = 0.001
EPS_DECAL = 0.05

OUTPUT_DIR = '/content/blob_saut2_results'
os.makedirs(OUTPUT_DIR, exist_ok=True)

# =====
# Classe BlobUniverse avec instrumentation T_n (temps emergent)
# =====
class BlobUniverse:
    """Identique au v2 + instrumentation T_n = somme D(C_k) dt_MCMC."""

    def __init__(self, seed=0):
        self.rng = np.random.default_rng(seed)
        self.seed = seed

        self.positions = [np.array([0.5, 0.5, 0.5])]
        self.adj = [set()]
        self.edge_list = []
        self.deg = np.array([0], dtype=np.int32)

        self.n_tri = 0
        self.n_tetra = 0
        self.S_curr = 0.0

        # Observables centrales du Saut 2
        self.Q3_dissipation = 0.0
        self.T_n_emergent = 0.0 # somme D dt_MCMC (temps emergent T1)

        self.n_mitoses = 0
        self.n_swaps_accepted = 0

        self.first_edge_step = None
        self.first_triangle_step = None
        self.first_tetrahedron_step = None

```

```

@property
def N(self):
    return len(self.positions)

@property
def E(self):
    return len(self.edge_list)

def _compute_D(self):
    if self.N == 0:
        return 0.0
    mu = float(self.deg.mean())
    if mu == 0:
        return 0.0
    return float(self.deg.std()) / mu

def _compute_S(self):
    return np.log(1 + self.n_tri) - BETA * self._compute_D()

def _rips_radius(self):
    if self.N <= 1:
        return 0.5
    c_3 = 4 * np.pi / 3
    return 1.5 * (KAPPA_0_TARGET / (c_3 * self.N)) ** (1.0/3)

def _count_tetra_through_edge(self, u, v):
    common = self.adj[u] & self.adj[v]
    if len(common) < 2:
        return 0
    common_list = list(common)
    count = 0
    for i, w in enumerate(common_list):
        adj_w = self.adj[w]
        for x in common_list[i+1:]:
            if x in adj_w:
                count += 1
    return count

def _recount_triangles(self):
    return sum(len(self.adj[u] & self.adj[v]) for u, v in self.edge_list) //

def _recount_tetrahedra(self):
    n = 0
    seen = set()
    for u, v in self.edge_list:
        common = self.adj[u] & self.adj[v]

```

```

        if len(common) < 2:
            continue
        common_list = sorted(common)
        for i, w in enumerate(common_list):
            for x in common_list[i+1:]:
                if x in self.adj[w]:
                    key = tuple(sorted([u, v, w, x]))
                    if key not in seen:
                        seen.add(key)
                        n += 1
    return n

def mitose(self):
    if self.N == 0:
        return
    parent_idx = int(self.rng.integers(self.N))
    parent_pos = self.positions[parent_idx]
    r_p = self._rips_radius()
    decal = self.rng.normal(0, EPS_DECAL * r_p, size=3)
    new_pos = parent_pos + decal
    self.positions.append(new_pos)
    self.adj.append(set())
    self.deg = np.append(self.deg, 0)
    self.n_mitoses += 1

def step(self, current_step):
    # Mitose
    if self.rng.random() < P_MITOSE:
        self.mitose()
        return None

    if self.N < 2:
        return None

    # Pas Blob (swap ou ajout pur si deficit)
    if self.E == 0:
        return self._step_add_only(current_step)
    E_target = int(KAPPA_0_TARGET * self.N / 2)
    if self.E < E_target:
        deficit_ratio = (E_target - self.E) / max(E_target, 1)
        if self.rng.random() < deficit_ratio:
            return self._step_add_only(current_step)
    return self._step_swap(current_step)

def _step_add_only(self, current_step):
    u = int(self.rng.integers(self.N))
    v = int(self.rng.integers(self.N))

```

```

        if u == v or v in self.adj[u]:
            return None
        u, v = min(u, v), max(u, v)
        dt_add = len(self.adj[u] & self.adj[v])
        self.adj[u].add(v)
        self.adj[v].add(u)
        self.deg[u] += 1
        self.deg[v] += 1
        new_n_tri = self.n_tri + dt_add
        dtetra_add = self._count_tetra_through_edge(u, v)
        new_S = np.log(1 + new_n_tri) - BETA * self._compute_D()
        delta_S = new_S - self.S_curr
        log_alpha = self.N * delta_S / TAU
        if log_alpha >= 0:
            accepted = True
        else:
            accepted = self.rng.random() < np.exp(max(log_alpha, -500))
        if accepted:
            self.edge_list.append((u, v))
            self.n_tri = new_n_tri
            self.n_tetra += dtetra_add
            self.S_curr = new_S
            self.n_swaps_accepted += 1
            self.Q3_dissipation += abs(delta_S)
            self.T_n_emergent += self._compute_D() # T1 : somme D dt_MCMC
            if self.first_edge_step is None and self.E > 0:
                self.first_edge_step = current_step
            if self.first_triangle_step is None and self.n_tri > 0:
                self.first_triangle_step = current_step
            if self.first_tetrahedron_step is None and self.n_tetra > 0:
                self.first_tetrahedron_step = current_step
            return abs(delta_S)
        else:
            self.adj[u].discard(v)
            self.adj[v].discard(u)
            self.deg[u] -= 1
            self.deg[v] -= 1
            return None

def _step_swap(self, current_step):
    idx = int(self.rng.integers(self.E))
    u_rm, v_rm = self.edge_list[idx]
    u_add = int(self.rng.integers(self.N))
    v_add = int(self.rng.integers(self.N))
    if u_add == v_add or v_add in self.adj[u_add]:
        return None
    u_add, v_add = min(u_add, v_add), max(u_add, v_add)

```

```

if (u_add, v_add) == (u_rm, v_rm):
    return None
dt_rm = len(self.adj[u_rm] & self.adj[v_rm])
dtetra_rm = self._count_tetra_through_edge(u_rm, v_rm)
self.adj[u_rm].discard(v_rm)
self.adj[v_rm].discard(u_rm)
self.deg[u_rm] -= 1
self.deg[v_rm] -= 1
dt_add = len(self.adj[u_add] & self.adj[v_add])
self.adj[u_add].add(v_add)
self.adj[v_add].add(u_add)
self.deg[u_add] += 1
self.deg[v_add] += 1
dtetra_add = self._count_tetra_through_edge(u_add, v_add)
new_n_tri = self.n_tri - dt_rm + dt_add
if new_n_tri < 0:
    self.adj[u_add].discard(v_add)
    self.adj[v_add].discard(u_add)
    self.deg[u_add] -= 1
    self.deg[v_add] -= 1
    self.adj[u_rm].add(v_rm)
    self.adj[v_rm].add(u_rm)
    self.deg[u_rm] += 1
    self.deg[v_rm] += 1
    return None
new_S = np.log(1 + new_n_tri) - BETA * self._compute_D()
delta_S = new_S - self.S_curr
log_alpha = self.N * delta_S / TAU
if log_alpha >= 0:
    accepted = True
else:
    accepted = self.rng.random() < np.exp(max(log_alpha, -500))
if accepted:
    self.edge_list[idx] = (u_add, v_add)
    self.n_tri = new_n_tri
    self.n_tetra += (dtetra_add - dtetra_rm)
    self.S_curr = new_S
    self.n_swaps_accepted += 1
    self.Q3_dissipation += abs(delta_S)
    self.T_n_emergent += self._compute_D()
    if self.first_triangle_step is None and self.n_tri > 0:
        self.first_triangle_step = current_step
    if self.first_tetrahedron_step is None and self.n_tetra > 0:
        self.first_tetrahedron_step = current_step
    return abs(delta_S)
else:
    self.adj[u_add].discard(v_add)

```

```

        self.adj[v_add].discard(u_add)
        self.deg[u_add] -= 1
        self.deg[v_add] -= 1
        self.adj[u_rm].add(v_rm)
        self.adj[v_rm].add(u_rm)
        self.deg[u_rm] += 1
        self.deg[v_rm] += 1
        return None

# =====
# RUN UNE SEED (avec instrumentation Q3 dense)
# =====
def run_seed_for_saut2(seed, total_steps):
    t_start = time.time()

    sim = BlobUniverse(seed=seed)

    # Echantillonnage logarithmique standard
    n_log_samples = 200
    log_sample_steps = sorted(set([
        max(1, int(np.exp(np.log(total_steps) * i / n_log_samples)))
        for i in range(1, n_log_samples + 1)
    ]))
    next_log = 0

    # Echantillonnage lineaire DENSE pour Q3 (1000 points)
    n_lin_samples = 1000
    lin_sample_steps = sorted(set([
        max(1, int(total_steps * i / n_lin_samples))
        for i in range(1, n_lin_samples + 1)
    ]))
    next_lin = 0

    trajectory = [] # echantillonnage logarithmique
    Q3_dense = [] # echantillonnage lineaire dense

    print(' Seed ' + str(seed) + ' init...', flush=True)

    for step in range(total_steps):
        sim.step(step + 1)

        # Echantillon logarithmique
        if next_log < len(log_sample_steps) and (step + 1) >= log_sample_steps[next_log]:
            traj_pt = {
                'step': step + 1,
                'N': sim.N, 'E': sim.E,

```

```

        'n_tri': sim.n_tri, 'n_tetra': sim.n_tetra,
        'D': sim._compute_D(),
        'Q3': sim.Q3_dissipation,
        'T_n': sim.T_n_emergent,
        'S': sim.S_curr,
        'n_swaps': sim.n_swaps_accepted,
        'time_s': time.time() - t_start,
    }
    trajectory.append(traj_pt)
    next_log += 1

    # Echantillon lineaire dense pour Q3
    if next_lin < len(lin_sample_steps) and (step + 1) >= lin_sample_steps[next_lin]:
        Q3_dense.append({
            'step': step + 1,
            'Q3': sim.Q3_dissipation,
            'T_n': sim.T_n_emergent,
            'n_swaps': sim.n_swaps_accepted,
        })
        next_lin += 1

total_time = time.time() - t_start

# Recount final
tri_recount = sim._recount_triangles()
tetra_recount = sim._recount_tetrahedra()
coherent = (sim.n_tri == tri_recount and sim.n_tetra == tetra_recount)

# =====
# ANALYSE Q3 - 5 PROPRIETES
# =====

# Construire serie Q3 dense
Q3_series = [pt['Q3'] for pt in Q3_dense]
T_n_series = [pt['T_n'] for pt in Q3_dense]
steps_series = [pt['step'] for pt in Q3_dense]

# P1 : Monotonie large (Q3[i+1] >= Q3[i])
n_violations_large = sum(1 for i in range(1, len(Q3_series))
                        if Q3_series[i] < Q3_series[i-1])
P1_large_monotonic = (n_violations_large == 0)

# P2 : Monotonie stricte (dQ3 > 0 sur intervalles non-triviaux)
# On accepte plateau seulement si E=0 (avant lere arete)
# Apres E>0, dQ3 > 0 strictement attendu
plateau_count = 0
plateau_after_edge = 0

```

```

if sim.first_edge_step:
    for i in range(1, len(Q3_series)):
        if Q3_series[i] == Q3_series[i-1]:
            plateau_count += 1
            if steps_series[i] > sim.first_edge_step:
                plateau_after_edge += 1
P2_strict_monotonic_post_edge = (plateau_after_edge == 0)

# P3 : Comportement asymptotique
# On regarde la derivee dQ3/dstep sur les 4 quartiles
n = len(Q3_series)
quartiles_dQ3 = []
for q in range(4):
    i_start = q * n // 4
    i_end = (q + 1) * n // 4 - 1
    if i_end > i_start:
        dQ3_q = (Q3_series[i_end] - Q3_series[i_start]) / (steps_series[i_end] - steps_series[i_start])
        quartiles_dQ3.append(float(dQ3_q))

# P4 : Decroissance de la derivee (concavite)
P4_concave = False
if len(quartiles_dQ3) == 4:
    # On veut q4 < q3 < q2 (decroissance) au moins sur les 3 derniers
    decreasing = quartiles_dQ3[3] <= quartiles_dQ3[2] <= quartiles_dQ3[1]
    # ou au moins q4 < q1 (signe global)
    ratio_last_first = quartiles_dQ3[3] / quartiles_dQ3[0] if quartiles_dQ3[0] != 0 else None
    P4_concave = decreasing or ratio_last_first < 0.5

# P5 : Correlation Q3 vs T_n
Q3_arr = np.array(Q3_series)
T_n_arr = np.array(T_n_series)
if len(Q3_arr) > 1 and Q3_arr.std() > 0 and T_n_arr.std() > 0:
    correlation_Q3_Tn = float(np.corrcoef(Q3_arr, T_n_arr)[0, 1])
else:
    correlation_Q3_Tn = 0.0

# =====
# RESUME
# =====
result = {
    'seed': seed,
    'total_steps': total_steps,
    'time_s': total_time,
    'final_state': {
        'N': sim.N, 'E': sim.E,
        'n_tri': sim.n_tri, 'n_tetra': sim.n_tetra,
        'Q3_final': sim.Q3_dissipation,
    }
}

```

```

        'T_n_final': sim.T_n_emergent,
        'D_final': sim._compute_D(),
        'n_swaps_accepted': sim.n_swaps_accepted,
    },
    'first_edge_step': sim.first_edge_step,
    'first_triangle_step': sim.first_triangle_step,
    'first_tetrahedron_step': sim.first_tetrahedron_step,
    'tri_recount': tri_recount,
    'tetra_recount': tetra_recount,
    'coherent': coherent,
    'saut2_analysis': {
        'P1_large_monotonic': P1_large_monotonic,
        'n_violations_large': n_violations_large,
        'P2_strict_monotonic_post_edge': P2_strict_monotonic_post_edge,
        'plateau_after_edge_count': plateau_after_edge,
        'P3_quartile_dQ3': quartiles_dQ3,
        'P4_concave': P4_concave,
        'P5_correlation_Q3_Tn': correlation_Q3_Tn,
    },
    'trajectory': trajectory,
    'Q3_dense': Q3_dense,
}

save_path = OUTPUT_DIR + '/saut2_seed' + str(seed) + '.json'
try:
    with open(save_path, 'w') as f:
        json.dump(result, f)
except Exception as e:
    print('  WARN save : ' + str(e))

# Resume console
print('  Seed ' + str(seed) + ' DONE en ' + '{:.1f}'.format(total_time/60) +
print('    Q3 final = ' + '{:.2f}'.format(sim.Q3_dissipation), flush=True)
print('    T_n emergent = ' + '{:.2f}'.format(sim.T_n_emergent), flush=True)
print('    P1 mono large : ' + str(P1_large_monotonic)
      + ' (' + str(n_violations_large) + ' violations)', flush=True)
print('    P2 mono stricte post-arete : ' + str(P2_strict_monotonic_post_edge)
      + ' (' + str(plateau_after_edge) + ' plateaux)', flush=True)
print('    P3 dQ3/dstep par quartile : '
      + str(['{:.2e}'.format(q) for q in quartiles_dQ3]), flush=True)
print('    P4 concave (ralentissement) : ' + str(P4_concave), flush=True)
print('    P5 correlation Q3 vs T_n : ' + '{:.4f}'.format(correlation_Q3_Tn),
print('    coherent : ' + str(coherent), flush=True)

del sim
gc.collect()
return result

```

```

# =====
# ANALYSE GLOBALE 5 SEEDS
# =====
def analyze_saut2(results):
    print('')
    print('=' * 75)
    print(' ANALYSE GLOBALE SAUT 2 - Q3 COMME FLECHE DU TEMPS')
    print('=' * 75)

    n_seeds = len(results)

    # Compter les seeds qui satisfont chaque propriete
    p1_count = sum(1 for r in results if r['saut2_analysis']['P1_large_monotonic'])
    p2_count = sum(1 for r in results if r['saut2_analysis']['P2_strict_monotonic'])
    p4_count = sum(1 for r in results if r['saut2_analysis']['P4_concave'])
    p5_corrs = [r['saut2_analysis']['P5_correlation_Q3_Tn'] for r in results]
    p5_count_high = sum(1 for c in p5_corrs if c > 0.95)

    # Stats Q3 final
    Q3_finals = [r['final_state']['Q3_final'] for r in results]
    T_n_finals = [r['final_state']['T_n_final'] for r in results]

    print('')
    print(' --- Resultats par seed ---')
    print(' Seed | P1 large | P2 stricte | P4 concave | P5 corr | Q3 final | T_n')
    print(' ' + '-' * 80)
    for r in results:
        s = r['seed']
        a = r['saut2_analysis']
        print(' ' + str(s) + ' | ' + str(a['P1_large_monotonic'])[0]
              + ' | ' + str(a['P2_strict_monotonic_post_edge'])[0]
              + ' | ' + str(a['P4_concave'])[0]
              + ' | ' + '{:.4f}'.format(a['P5_correlation_Q3_Tn'])
              + ' | ' + '{:>8.2f}'.format(r['final_state']['Q3_final'])
              + ' | ' + '{:>9.2f}'.format(r['final_state']['T_n_final']))

    # =====
    # TESTS DE VALIDATION
    # =====
    print('')
    print('=' * 75)
    print(' TESTS DE VALIDATION')
    print('=' * 75)

    tests = {}

```

```

# T1 : monotonie large 5/5
test1_pass = (p1_count == n_seeds)
tests['T1_large_monotonic'] = {
    'description': 'P1 monotonie large sur toutes les seeds',
    'count': p1_count,
    'pass': test1_pass
}
print('  T1 - Monotonie large (P1)          : ' + str(p1_count) + '/' + str
      + (' PASS' if test1_pass else ' FAIL'))

# T2 : monotonie stricte post-arete 5/5
test2_pass = (p2_count == n_seeds)
tests['T2_strict_monotonic'] = {
    'description': 'P2 monotonie stricte post-arete 5/5',
    'count': p2_count,
    'pass': test2_pass
}
print('  T2 - Monotonie stricte post-arete   : ' + str(p2_count) + '/' + str
      + (' PASS' if test2_pass else ' FAIL'))

# T3 : ralentissement (concavite)
test3_pass = (p4_count >= n_seeds - 1) # tolerer 1 echec
tests['T3_concave'] = {
    'description': 'P4 ralentissement (>= 4/5 seeds)',
    'count': p4_count,
    'pass': test3_pass
}
print('  T3 - Ralentissement (concavite)     : ' + str(p4_count) + '/' + str
      + (' PASS' if test3_pass else ' FAIL'))

# T4 : correlation Q3 vs T_n > 0.95 sur 5/5
test4_pass = (p5_count_high == n_seeds)
p5_min = min(p5_corrs) if p5_corrs else 0
p5_mean = np.mean(p5_corrs) if p5_corrs else 0
tests['T4_correlation'] = {
    'description': 'P5 correlation Q3 vs T_n > 0.95 sur 5/5',
    'count_high': p5_count_high,
    'min_corr': float(p5_min),
    'mean_corr': float(p5_mean),
    'pass': test4_pass
}
print('  T4 - Correlation Q3 vs T_n > 0.95   : ' + str(p5_count_high) + '/'
      + ' (min=' + '{:.4f}'.format(p5_min) + ', mean=' + '{:.4f}'.format(p5_m
      + (' PASS' if test4_pass else ' FAIL'))

# =====

```

```

# FALSIFICATIONS
# =====

print('')
print(' --- Falsifications potentielles ---')

# F1 : Q3 sature (dQ3/dt -> 0 en fin)
saturated = []
for r in results:
    q = r['saut2_analysis']['P3_quartile_dQ3']
    if len(q) == 4 and q[0] > 0 and q[3] / q[0] < 0.05:
        saturated.append(r['seed'])
f1_triggered = len(saturated) > 0
print(' F1 - Q3 sature (q4/q1 < 5%) : '
      + str(len(saturated))
      + (' DECLENCHE pour seeds ' + str(saturated) if f1_triggered else ' neg

# F2 : Q3 non-monotone (au moins 1 violation)
violators = [r['seed'] for r in results
              if r['saut2_analysis']['n_violations_large'] > 0]
f2_triggered = len(violators) > 0
print(' F2 - Q3 non-monotone : '
      + str(len(violators))
      + (' DECLENCHE pour seeds ' + str(violators) if f2_triggered else ' neg

# F3 : correlation Q3 vs T_n < 0.5 sur au moins 1 seed
low_corr = [r['seed'] for r in results
            if r['saut2_analysis']['P5_correlation_Q3_Tn'] < 0.5]
f3_triggered = len(low_corr) > 0
print(' F3 - Correlation Q3 vs T_n < 0.5 : '
      + str(len(low_corr))
      + (' DECLENCHE pour seeds ' + str(low_corr) if f3_triggered else ' nega

# =====
# VERDICT
# =====

print('')
print('=' * 75)
print(' VERDICT SAUT 2 - Q3 = FLECHE DU TEMPS')
print('=' * 75)

n_pass = sum(1 for t in tests.values() if t.get('pass'))
n_total = len(tests)
no_falsif = not (f1_triggered or f2_triggered or f3_triggered)

full_M = (n_pass == n_total) and no_falsif
partial_M = (n_pass >= n_total - 1) and (not f2_triggered)

```

```

print(' Tests passes : ' + str(n_pass) + '/' + str(n_total))
print('')

if full_M:
    print(' >> VALIDATION COMPLETE [M]')
    print(' >> Q3 satisfait toutes les proprietes d une fleche du temps')
    print(' >> CANDIDAT [T] : Q3 = somme |Delta S| -> monotone par construct')
    print(' >> Statut Q3 : passe de [I] a [M] verrouille')
elif partial_M:
    print(' >> VALIDATION PARTIELLE [M]')
    print(' >> Reformulation possible des criteres en echec')
else:
    print(' >> ECHEC ou MIXED')
    print(' >> Q3 n est pas une fleche du temps standard')

return {
    'n_seeds': n_seeds,
    'tests': tests,
    'falsifications': {
        'F1_saturated': {'count': len(saturated), 'seeds': saturated, 'trigge
        'F2_non_monotonic': {'count': len(violators), 'seeds': violators, 'tr
        'F3_low_correlation': {'count': len(low_corr), 'seeds': low_corr, 'tr
    },
    'verdict': {
        'tests_passed': n_pass,
        'tests_total': n_total,
        'full_validation_M': full_M,
        'partial_validation_M': partial_M,
    },
    'stats': {
        'Q3_finals': Q3_finals,
        'T_n_finals': T_n_finals,
        'correlations_Q3_Tn': p5_corrs,
    }
}

# =====
# MAIN
# =====

print('')
print('=' * 75)
print(' BLOB PRINCIPLE - SAUT 2 - Q3 = FLECHE DU TEMPS')
print('=' * 75)
print(' Objectif : caracteriser Q3 = somme |Delta S| comme fleche du temps')
print(' Proprietes testees :')
print(' P1 - Monotonie large (Q3[i+1] >= Q3[i])')

```

```

print('    P2 - Monotonie STRICTE post-1ere arete')
print('    P3 - Comportement asymptotique (4 quartiles)')
print('    P4 - Concavite (ralentissement)')
print('    P5 - Correlation Q3 vs T_n emergent (T1)')
print('=' * 75)

TOTAL_STEPS = int(os.environ.get('BLOB_TOTAL_STEPS', 5000000))
SEEDS_TO_RUN = [int(s) for s in os.environ.get('BLOB_SEEDS', '0,1,2,3,4').split('

print('')
print(' Configuration :')
print('    Pas par seed : ' + str(TOTAL_STEPS))
print('    Seeds : ' + str(SEEDS_TO_RUN))
print('    Duree estimee : ~' + str(7 * len(SEEDS_TO_RUN)) + ' min')
print('=' * 75)

t_global = time.time()
all_results = []

for seed in SEEDS_TO_RUN:
    print('')
    print('-' * 75)
    print(' Seed ' + str(seed) + ' (' + str(SEEDS_TO_RUN.index(seed) + 1)
          + '/' + str(len(SEEDS_TO_RUN)) + ')')
    print('-' * 75)
    try:
        r = run_seed_for_saut2(seed, TOTAL_STEPS)
        all_results.append(r)
    except Exception as e:
        print(' ERREUR seed=' + str(seed) + ' : ' + str(e))
        import traceback
        traceback.print_exc()

global_time = time.time() - t_global

if len(all_results) >= 2:
    analysis = analyze_saut2(all_results)

    report_path = OUTPUT_DIR + '/saut2_RAPPORT.json'
    try:
        with open(report_path, 'w') as f:
            json.dump({
                'config': {
                    'total_steps': TOTAL_STEPS,
                    'seeds': SEEDS_TO_RUN,
                    'global_time_s': global_time,
                },

```

```

        'analysis': analysis
    }, f, indent=2)
    print('')
    print(' Rapport final : ' + report_path)
except Exception as e:
    print(' WARN save report : ' + str(e))

print('')
print('=' * 75)
print('  CAMPAGNE SAUT 2 TERMINEE')
print('=' * 75)
print(' Temps global : ' + '{:.1f}'.format(global_time/60) + ' min')
print(' Fichiers dans : ' + OUTPUT_DIR)
for f in sorted(os.listdir(OUTPUT_DIR)):
    sz = os.path.getsize(OUTPUT_DIR + '/' + f)
    print('      ' + f + ' (' + str(sz) + ' octets)')
print('=' * 75)

```